

OOPS

Private attributes & methods

`__` → private - u cannot access outside the class

`_` → protected

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    # Getter
    @property
    def radius(self):
        return self._radius

    # Setter
    @radius.setter
    def radius(self, value):
        if value <= 0:
            raise ValueError("Radius must be positive")
        self._radius = value

    # Deleter
    @radius.deleter
    def radius(self):
        print("Deleting radius...")
        del self._radius
```

Inheritance

```

class Animal:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return f'{self.name} makes a sound.'

class Dog(Animal):
    bark = 'woof! woof!! woof!!!'

    # Override sound() to use bark class variable
    def sound(self):
        return f'{self.name} barks{self.bark}'

jack = Dog('Jack')
print(jack.sound()) # Jack barks woof! woof!! woof!!!

```

```

class Animal:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return f'{self.name} makes a sound'

class Dog(Animal):
    bark = 'woof! woof!! woof!!!'

    # Call Animal.sound(), then append bark
    def sound(self):
        base = super().sound()
        return f'{base}, then{self.name} barks{self.bark}'

jack = Dog('Jack')

```

```
print(jack.sound()) # Jack makes a sound, then Jack barks wo  
of! woof!! woof!!!
```

Polymorphism -

```
class Cat:  
    def speak(self):  
        return "A cat meow"  
  
class Bird:  
    def speak(self):  
        return "A bird tweet"  
  
class Monkey:  
    def speak(self):  
        return "A monkey ooh ooh aah aah ooh ooh aah aah"  
  
def animal_sound(animal):  
    print(animal.speak())  
  
animal_sound(Cat())  
animal_sound(Bird())  
animal_sound(Monkey())
```

Inheritance-based polymorphism

In inheritance-based polymorphism, a parent class defines a method, and multiple child classes override that method in their own way. You can then call the same method on any child object, and it behaves differently depending on which child class it is.

```
class Animal:
    def speak(self):
        return 'Some generic sound'

class Cat(Animal):
    def speak(self):
        return 'A cat meow'

class Dog(Animal):
    def speak(self):
        return 'A dog barks woof woof'

class Monkey(Animal):
    def speak(self):
        return 'A monkey ooh ooh aah aah ooh ooh aah aah'

print(Cat().speak()) # A cat meow
print(Dog().speak()) # A dog barks woof woof
print(Monkey().speak()) # A monkey ooh ooh aah aah ooh ooh aa
h aah
print(Animal().speak()) # Some generic sound
```

Name Mangling -